

Thuật giải heuristic

PGS.TS. Nguyễn Đình Hiền

1

THUẬT GIẢI HEURISTIC

1. Giới thiệu:

- + Khái niệm
- + Các nguyên lý

2. Áp dụng:

- + Bài toán phân công
- + Bài toán tô màu đồ thị
- + Bài toán người đưa thư

2

Mục tiêu bài học:

- Hiểu được thuật giải Heuristic
- Hiểu được các nguyên lý của thuật giải Heuristic
- Áp dụng các thuật giải Heuristic trong một số bài toán ứng dụng.

3

KHÁI NIỆM THUẬT GIẢI HEURISTIC

4

Thuật toán

Là giải pháp được viết dưới **dạng thủ tục** và thoả các tiêu chuẩn:

Tính xác định: Không mập mờ và có thể thực thi được

Tính hữu hạn: chương trình phải dừng

Tính đúng: cho kết quả chính xác

5

Thuật giải

Là giải pháp được viết dưới dạng thủ tục tương tự như thuật toán nhưng **không đòi hỏi** các tiêu chuẩn như thuật toán.

Tính đúng: chấp nhận các **thuật giải đơn giản** có thể cho **kết quả đúng** hay **gần đúng** nhưng có **khả năng thành công** cao hơn.

6

Để có thể được chấp nhận, thuật giải phải thể hiện một giải pháp hợp lý nhất có thể trong tình huống hiện tại bằng cách:

 **Tận dụng** mọi thông tin hữu ích

 Sử dụng **tri thức, kinh nghiệm** trực giác của con người

 Tự nhiên **đơn giản** nhưng cho **kết quả chấp nhận được**

➔ **THUẬT GIẢI HEURISTIC**

7

Thuật giải heuristic

Các đặc tính:

- Thường tìm được **lời giải tốt**, mặc dù không phải là tốt nhất
- Có **độ phức tạp nhỏ** hơn so với **thuật giải tối ưu**
- Khá **tự nhiên gần gũi** với cách giải của con người

8



9

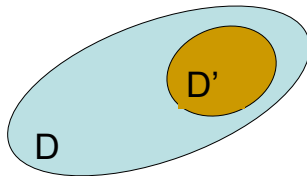
CÁC NGUYÊN LÝ

- ➡ Nguyên lý vét cạn thông minh
- ➡ Nguyên lý tham lam
- ➡ Nguyên lý thứ tự
- ➡ Hàm Heuristic

10

➡ Nguyên lý vét cạn thông minh

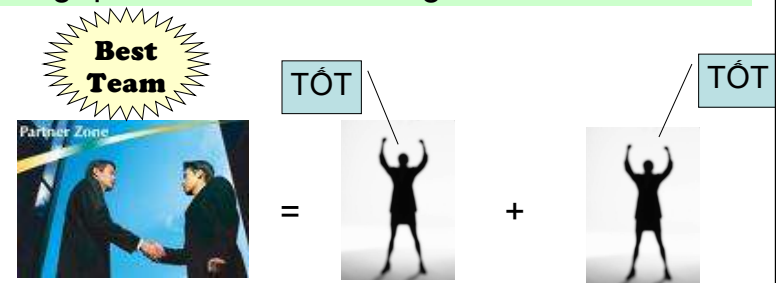
Trong một bài toán tìm kiếm nào đó, khi không gian tìm kiếm lớn, ta thường tìm cách giới hạn lại không gian tìm kiếm hoặc thực hiện một kiểu dò tìm đặc biệt dựa vào đặc thù của bài toán để nhanh chóng tìm ra mục tiêu.



11

➡ Nguyên lý tham lam

Lấy tiêu chuẩn **tối ưu** (trên phạm vi **toàn cục**) của bài toán, dựa vào đó **chọn** lựa **hành động tốt nhất** của **từng bước** (hay **từng giai đoạn**) trong quá trình tìm kiếm lời giải.



12



Nguyên lý thứ tự

Là thực hiện hành động dựa trên một **cấu trúc thứ tự hợp lý** của không gian khảo sát nhằm nhanh chóng đạt được lời giải tốt.

A B C D E F

13



Hàm Heuristic

Trong việc xây dựng các thuật giải Heuristic, người ta thường dùng các **hàm Heuristic**. Đó là các **hàm lượng giá** các giá trị, **phụ thuộc vào trạng thái hiện tại** của bài toán tại mỗi bước giải. Nhờ giá trị này, ta có thể **chọn** được cách **hành động tương đối hợp lý** trong từng bước của thuật giải.



Hàm heuristic h

14

BÀI TOÁN

PHÂN CÔNG CÔNG VIỆC

15

Phát biểu bài toán:

Một đề án gồm n **công việc** và các việc sẽ được thực hiện bởi m **máy** như nhau.

Giả sử biết thời gian để 1 máy thực hiện việc thứ j là t_j

Yêu cầu: Tìm phương án phân công sao cho **thời gian** hoàn thành toàn bộ công việc là **thấp nhất**.

Ví dụ:

$n=10, m=3$

$t_j = 4 \ 9 \ 5 \ 2 \ 7 \ 6 \ 10 \ 8 \ 7 \ 5$

16

Có n công việc, phân công cho m máy. Khi đó thuật toán tối ưu sẽ có độ phức tạp là $O(m^n)$



→ Vì thế ta phải tìm một **thuật giải Heuristic** cho bài toán phân công công việc

17

Dựa theo kinh nghiệm con người:

Ta sẽ lấy **công việc** có thời gian **lớn nhất** phân công cho **máy** có thời gian **làm việc ít**.



→ Nguyên lý thứ tự

18

Thuật giải cho bài toán phân công đơn giản:

```
for (k=0;k<n;k++)
{
    + Chọn việc J chưa phân công có thời gian thực hiện cao nhất.
    + Chọn máy M có thời gian làm việc thấp nhất
    + Bố trí việc J cho máy M.
}
```

19

Ví dụ:

$n=10, m=3$

tj = 4 9 5 2 7 7 6 1 6 8 7 2

M1	10	6	5	
M2	9	7	4	2
M3	8	7	5	

20

Tổ chức dữ liệu chương trình:

```
int n, m // n công việc, m máy
```

```
int t[100] // t[j] là thời gian thực hiện công việc j
```

```
int id[100] // Tên các công việc
```

```
int machine[100] // th.gian làm việc của các máy
```

```
int kq[100] // công việc j sẽ được phân công  
cho kq[j]
```

```
int ChonViec( ) // Hàm chọn công việc
```

```
int ChonMachine( ) // Hàm chọn máy
```

21

Thuật giải Heuristic cho bài toán phân công công việc có độ phức tạp là $O(n \lg n + m^2)$

=> So sánh với $O(m^n)$

Tuy nhiên trong một số trường hợp thuật giải chưa cho ra lời giải chính xác.

Ví dụ:

$n=5, m=2$

$t_j = 3 \ 3 \ 2 \ 2 \ 2$

22

BÀI TOÁN

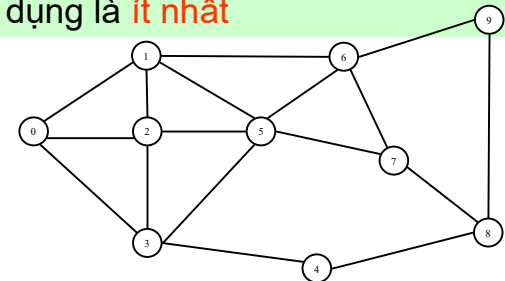
TÔ MÀU ĐỒ THỊ

23

Phát biểu bài toán:

Cho một đồ thị vô hướng đơn giản. Ta muốn tìm cách tô màu cho các đỉnh của đồ thị sao cho 2 đỉnh cạnh nhau phải có màu khác nhau.

Yêu cầu: Tìm phương án tô sao cho số màu sử dụng là ít nhất



24

Sử dụng nguyên lý thứ tự:

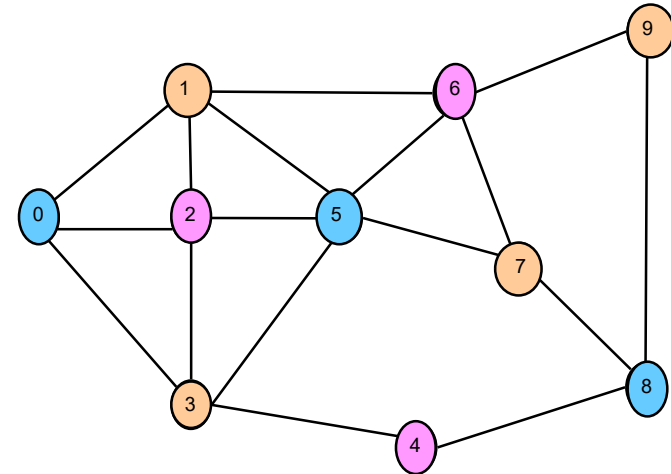
```
for (k=0;k<n;k++)
{
    + Chọn đỉnh S: có số đỉnh chưa tô ở
    cạnh nó là lớn nhất (bậc lớn nhất)

    + Chọn màu: Ưu tiên tô đỉnh S bằng
    những màu đã tô, nếu không được thì sử
    dụng màu mới

    + Sau khi tô màu cho đỉnh S thì ta xóa các
    cạnh có nối đến S và đánh dấu các đỉnh
    kề bên không được tô màu vừa tô cho S
}
```

25

Ví dụ Đỉnh: 0 1 2 3 4 5 6 7 8 9
Bậc: -1 -1 -1 -1 -1 -1 -1 -1 -1 -1



26

Tổ chức dữ liệu chương trình:

```
int n, m // n đỉnh, m màu
int DT[100][100] // Đồ thị dưới dạng ma trận
int bậc[100] // bậc của các đỉnh
int Mau[100] // Đỉnh i được tô màu Mau[ i ]

int ChonDinh( ) // Hàm chọn đỉnh để tô
int ChonMau( int S ) // Hàm chọn màu đỉnh S
int XuLy( int S ) // Hàm giảm bậc các đỉnh kề
của đỉnh S
```

27

int ChonMau(int S) // Hàm chọn màu đỉnh S

```
{
    for(color = 0;color<m;color++)
    {
        OK = true;
        for (j=0;j<n;j++)
            if (DT[S][j] !=0 // Đỉnh j có kề S
                && Mau[j] ==color) // Đỉnh j có là color
                OK = false;
        if (OK) return color;
    }
    return (++m);
}
```

28

Ứng dụng:

+ Sắp lịch thi học kì



+ Sắp lịch hội thảo

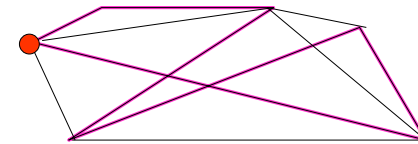


.....

29

BÀI TOÁN

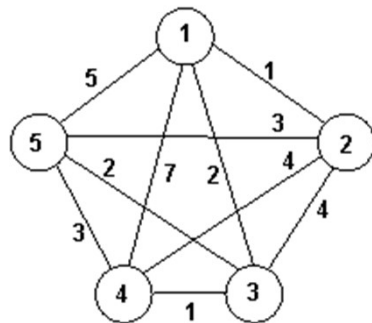
Người du lịch (Travel Saleman Problem)



30

Phát biểu bài toán:

Giả sử có một đồ thị có trọng số dương, tìm **đường đi ngắn nhất** qua **tất cả** các đỉnh của đồ thị rồi **trở về đỉnh ban đầu**



31

Đồ thị có n đỉnh, khi đó thuật toán tối ưu sẽ là thuật toán tìm chu trình Hamilton ngắn nhất, có độ phức tạp là $O(n!)$



→ Vì thế ta phải tìm một **thuật giải Heuristic** cho bài toán người đưa thư.

32

Dựa theo kinh nghiệm con người:

Khi đi trên **những đoạn đường ngắn nhất** thì cuối cùng ta sẽ có một **hành trình ngắn nhất**.



⇒ Nguyên lý tham lam

33

Thuật giải sử dụng nguyên lý tham lam:

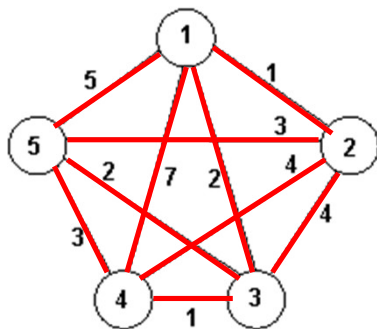
While (Chưa đi hết các đỉnh)

{
 Khi đã đi đến một đỉnh, chọn đi đến đỉnh kế tiếp theo nguyên tắc:

 ✱ Liệt kê tất cả con đường từ đỉnh hiện tại đến những đỉnh chưa đi đến. Chọn con đường ngắn nhất.

}

34



Chiều dài đường đi là: 1 + 3 + 2 + 1 + 7

= 14

35

Tổ chức dữ liệu chương trình:

int **n** // n đỉnh

int **DT[100][100]** // Đồ thị dưới dạng ma trận

int **Path[100]** // Đỉnh i sẽ đi đến đỉnh Path[i]

int **Ke(int S)** // Tìm đỉnh đi kế tiếp sau đỉnh S

int **Tim(int S)** // Tìm đường đi bắt đầu từ S

36

```

int Tim( int S ) //Tìm đường đi từ đỉnh S
{
    e = S; dem = 1;
    while (dem < n)
    {
        k = Ke(e);
        Path[e] = k;
        e = k;
        dem ++;
    }
}

```

37

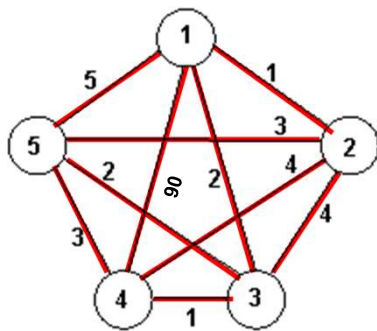
Thuật giải Heuristic cho bài toán người đưa thư có độ phức tạp là $O(n^2)$

=> So sánh với $O(n!)$

Tuy nhiên thuật giải có những hạn chế, chưa cho ra lời giải chính xác.

38

Ví dụ:



Chiều dài đường đi là: 1 + 3 + 2 + 1 + 90
= 97

39

Lời giải tốt, chứ
không phải tốt nhất

Sử dụng kinh nghiệm
con người

Hiệu quả

THUẬT GIẢI
HEURISTIC

Vết cận
thông minh

Tham lam

Thứ tự

Hàm
Heuristic

40

HOMEWORK

- 🌐 Cài đặt bài toán phân công công việc, tô màu đồ thị.
- 🌐 Bài toán phân công công việc, trong đó các công việc có ràng buộc trình tự.
- 🌐 Cài đặt bài toán sắp lịch thi học kì.
- 🌐 Bài toán người đưa thư theo 2 cách: vét cạn và sử dụng thuật giải Heuristic.
- 🌐 Xem lại thuật giải Dijkstra (chuẩn bị bài cho kì tới)